

```

1 import components.naturalnumber.NaturalNumber;
2
3
4 /**
5  * Controller class.
6  *
7  * @author Eric Zhou
8  */
9 public final class NNCalcController1 implements NNCalcController {
10
11     /**
12      * Model object.
13      */
14     private final NNCalcModel model;
15
16     /**
17      * View object.
18      */
19     private final NNCalcView view;
20
21     /**
22      * Useful constants.
23      */
24     private static final NaturalNumber TWO = new NaturalNumber2(2),
25         INT_LIMIT = new NaturalNumber2(Integer.MAX_VALUE);
26
27     /**
28      * Updates this.view to display this.model, and to allow only operations
29      * that are legal given this.model.
30      *
31      * @param model
32      *         the model
33      * @param view
34      *         the view
35      * @ensures [view has been updated to be consistent with model]
36      */
37     private static void updateViewToMatchModel(NNCalcModel model, NNCalcView view) {
38
39         view.updateTopDisplay(model.top());
40         view.updateBottomDisplay(model.bottom());
41         view.updateSubtractAllowed(model.bottom().compareTo(model.top()) <= 0);
42         view.updateDivideAllowed(!model.bottom().isZero());
43         view.updatePowerAllowed(model.bottom().compareTo(INT_LIMIT) <= 0);
44         view.updateRootAllowed(model.bottom().compareTo(TWO) >= 0
45             && model.bottom().compareTo(INT_LIMIT) <= 0);
46     }
47 }
48
49 /**
50  * Constructor.
51  *
52  * @param model
53  *         model to connect to
54  * @param view
55  *         view to connect to
56  */
57 public NNCalcController1(NNCalcModel model, NNCalcView view) {
58     this.model = model;
59     this.view = view;
60     updateViewToMatchModel(model, view);
61 }
62
63 @Override
64 public void processClearEvent() {
65     /**
66      * Get alias to bottom from model

```

```

67    /**
68     * NaturalNumber bottom = this.model.bottom();
69     */
70     * Update model in response to this event
71     */
72     bottom.clear();
73     /*
74     * Update view to reflect changes in model
75     */
76     updateViewToMatchModel(this.model, this.view);
77 }
78
79 @Override
80 public void processSwapEvent() {
81     /*
82     * Get aliases to top and bottom from model
83     */
84     NaturalNumber top = this.model.top();
85     NaturalNumber bottom = this.model.bottom();
86     /*
87     * Update model in response to this event
88     */
89     NaturalNumber temp = top.newInstance();
90     temp.transferFrom(top);
91     top.transferFrom(bottom);
92     bottom.transferFrom(temp);
93     /*
94     * Update view to reflect changes in model
95     */
96     updateViewToMatchModel(this.model, this.view);
97 }
98
99 @Override
100 public void processEnterEvent() {
101     NaturalNumber top = this.model.top();
102     top.transferFrom(this.model.bottom()); //copy the content
103     updateViewToMatchModel(this.model, this.view);
104
105 }
106
107 @Override
108 public void processAddEvent() {
109
110     NaturalNumber top = this.model.top();
111     this.model.bottom().add(top);
112     top.clear(); //clear the top part
113     updateViewToMatchModel(this.model, this.view);
114
115 }
116
117 @Override
118 public void processSubtractEvent() {
119
120     NaturalNumber top = this.model.top();
121     NaturalNumber bottom = this.model.bottom();
122     top.subtract(bottom);
123     bottom.transferFrom(top);
124     top.clear();
125     updateViewToMatchModel(this.model, this.view);
126
127 }
128
129 @Override
130 public void processMultiplyEvent() {
131

```

```

132 NaturalNumber top = this.model.top();
133 NaturalNumber bottom = this.model.bottom();
134 top.multiply(bottom);
135 bottom.transferFrom(top);
136 top.clear();
137 updateViewToMatchModel(this.model, this.view);
138
139 }
140
141 @Override
142 public void processDivideEvent() {
143
144     NaturalNumber top = this.model.top();
145     NaturalNumber bottom = this.model.bottom();
146     top.divide(bottom); // top = top / bottom
147     bottom.transferFrom(top); // bottom = result
148     top.clear(); // top = 0
149     updateViewToMatchModel(this.model, this.view);
150
151 }
152
153 @Override
154 public void processPowerEvent() {
155
156     NaturalNumber topNum = this.model.top();
157     NaturalNumber bottomNum = this.model.bottom();
158     /*
159      * Update model in response to this event
160      */
161     topNum.power(bottomNum.toInt());
162     bottomNum.transferFrom(topNum);
163     /*
164      * Update view to reflect changes in model
165      */
166     updateViewToMatchModel(this.model, this.view);
167
168 }
169
170 @Override
171 public void processRootEvent() {
172
173     NaturalNumber topNum = this.model.top();
174     NaturalNumber bottomNum = this.model.bottom();
175     /*
176      * Update model in response to this event
177      */
178     topNum.root(bottomNum.toInt());
179     bottomNum.transferFrom(topNum);
180     /*
181      * Update view to reflect changes in model
182      */
183     updateViewToMatchModel(this.model, this.view);
184
185 }
186
187 @Override
188 public void processAddNewDigitEvent(int digit) {
189
190     this.model.bottom().multiplyBy10(digit); //use multiplyBy10 to add digit
191     updateViewToMatchModel(this.model, this.view);
192
193 }
194
195 }
196

```